

Name: Jiya Yogesh Varma

PRN: 202501050023

Division: Chemical

Batch: CH2

Course: EDS



1.1.1. Calculate Momentum 04:41

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:
 m is the mass of the object (in kilograms).
 v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

Test case 1

5.0 10.0 50.00kgm/s

Test case 2

2.3 4.8 11.04kgm/s

calculateM...

```
1 mass = float(input())
2 velocity = float(input())
3 momentum = mass * velocity
4 print(f"{momentum:.2f}kgm/s")
```

Average time	Maximum time
0.031 s	0.039 s
30.75 ms	39.00 ms

2 out of 2 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1

39 ms

Debug

Expected output

Actual output

5.0

5.0

10.0

10.0

50.00kgm/s

50.00kgm/s



1.1.2. Conditional Calculation Based on the ... 03:52

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

$1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

Test case 1

9

81

Test case 2

166

5.50

Test case 3

24

4.90

Test case 4

3456

Invalid

Explorer

conditional...



Submit

Debugger

```
1 import math
2
3 number = int(input())
4
5 if 1 <= number <= 9:
6     print(number ** 2)
7
8 elif 10 <= number <= 99:
9     print(f"{math.sqrt(number):.2f}")
10
11 elif 100 <= number <= 999:
12     print(f"{number ** (1/3):.2f}")
13
14 else:
15     print("Invalid")
```

Average time

0.017 s

17.29 ms

Maximum time

0.021 s

21.00 ms

4 out of 4 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 21 ms

Debug



Expected output

Actual output

9

9

81

81

< Prev

Reset

Submit

Next >



1.1.3. Days between Two Dates

15:40

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

Test case 1

2014-07-02

2014-11-02

123

Test case 2

2014-07-02

2014-07-11

9

```
noOfDays.py
1 from datetime import date
2 from datetime import datetime
3
4 date1=input().strip()
5 date2=input().strip()
6
7 d1=datetime.strptime(date1,"%Y-%m-%d")
8 d2=datetime.strptime(date2,"%Y-%m-%d")
9
10 difference=d2-d1
11
12 print(difference.days)
```

Average time	Maximum time
0.038 s	0.064 s
38.25 ms	64.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1	64 ms	Debug			
Expected output	Actual output				
2014-07-02	2014-07-02				
2014-11-02	2014-11-02				
123	123				



1.1.4. Reverse a Number 02:06

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

Test case 1

5367

7635

Test case 2

0132

231

```
reverseNu... Submit
1 #write your code here...
2 num = int(input())
3 reversed_num = int(str(num)
  [::-1])
4 print(reversed_num)
```

Average time 0.022 s 21.80 ms Maximum time 0.027 s 27.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1 20 ms Debug

Expected output Actual output

5367 5367

7635 7635



1.1.5. Multiplication Table

00:39



Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

<number> x <multiplier> = <result>

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

Test case 1

8	
8 x 1 = 8	
8 x 2 = 16	
8 x 3 = 24	
8 x 4 = 32	
8 x 5 = 40	
8 x 6 = 48	
8 x 7 = 56	
8 x 8 = 64	
8 x 9 = 72	
8 x 10 = 80	

Test case 2

5	
5 x 1 = 5	
5 x 2 = 10	
5 x 3 = 15	
5 x 4 = 20	
5 x 5 = 25	
5 x 6 = 30	
5 x 7 = 35	
5 x 8 = 40	
5 x 9 = 45	
5 x 10 = 50	

Explorer

multiplicat...



Submit

```
1 #write your code here...
2 num = int(input())
3
4 for i in range(1,11):
5     print(f"{num} x {i} = {num * i}")
6
7
```

Debugger

Average time

0.018 s

18.25 ms

Maximum time

0.025 s

25.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 18 ms

Debug



Expected output

Actual output

8

8

8 x 1 = 8

8 x 1 = 8

8 x 2 = 16

8 x 2 = 16



< Prev

Reset

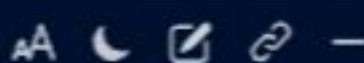
Submit

Next >



1.2.1. Pass or Fail

00:17



Write a Python program that accepts the number of courses and the marks of a student in those courses. The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases**Test case 1**

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2

5

56 78 97 86 93

Aggregate Percentage: 82.00

Grade: Distinction

Test case 3

5

99 85 43 97 34

Fail

Test case 4

3

55 54 53

Aggregate Percentage: 54.00

Explorer

passorFail....



Submit

Debugger

```
1 # write your code here
2 def cal(marks, total_courses):
3     if any(mark < 40 for mark
4         in marks):
5         return "Fail"
6     total_marks = sum(marks)
7     aggregate_percentage =
8         (total_marks / (total_courses
9             * 100)) * 100
10
11     if aggregate_percentage >
12         75:
13         grade = "Distinction"
14     elif aggregate_percentage
15         >= 60:
16         grade = "First
17             Division"
18     elif aggregate_percentage
19         >= 50:
20         grade = "Second
21             Division"
22     elif aggregate_percentage
23         >= 40:
24         grade = "Third
25             Division"
26     return
27     (aggregate_percentage, grade)
28
29 num_courses = int(input())
30 marks = list(map(int,
31     input().split()))
32
33 result = cal(marks,
34     num_courses)
35
36 if result == "Fail":
37     print("Fail")
38 else:
39     aggregate_percentage,
40     grade = result
41     print(f"Aggregate
42         Percentage:
43         {aggregate_percentage:.2f}")
44     print(f"Grade: {grade}")
```

Average time

0.029 s

28.90 ms

Maximum time

0.042 s

42.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 42 ms

Debug



Expected output

Actual output

4

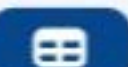
4

55 65 78 89

55 65 78 89

Aggregate Percentage: 71.75

Aggregate Percentage: 71.75



< Prev

Reset

Submit

Next >



1.2.2. Fibonacci Series

14:33



Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

Test case 1

5

0 1 1 2 3

Test case 2

15

0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Explorer

fibonacci.py



Submit

```
1 def fibonacci(n):
2
3     if n==1:
4         return 0
5     elif n==2:
6         return 1
7     else:
8         return fibonacci(n-1)
9         + fibonacci(n-2)
10    n = int(input())
11    for i in range(1, n + 1):
12        print(fibonacci(i), end="")
```

Debugger

Average time

0.036 s

35.75 ms

Maximum time

0.082 s

82.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

20 ms

Debug



Expected output

5

0 1 1 2 3

Actual output

5

0 1 1 2 3

< Prev

Reset

Submit

Next >



Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

Test case 1

5

* .

* . * .

* . * . * .

* . * . * . * .

* . * . * . * . * .

Test case 2

4

* .

* . * .

* . * . * .

* . * . * . * .

```
1 #Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     for j in range(i):
5         print("*",end=" ")
6     print()
7
```

Average time

0.018 s

17.83 ms

Maximum time

0.026 s

26.00 ms

2 out of 2 shown test case(s) passed

4 out of 4 hidden test case(s) passed

Test case 1 26 ms

Debug

Expected output

Actual output

5

5

* .

* .

* . * .

* . * .



Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases

Test case 1

5

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

Test case 2

8

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

1 2 3 4 5 6

1 2 3 4 5 6 7

1 2 3 4 5 6 7 8

Explorer

numberPat...



Submit

```
1 # Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     for j in range(1,i+1):
5         print(j,end=" ")
6     print()
7
```

Debugger

Average time

0.026 s

25.75 ms

Maximum time

0.043 s

43.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1 43 ms

Debug



Expected output

5

1

1 2

Actual output

5

1

1 2

